MALACKATHON 2026 · UNIVERSIDAD DE MÁLAGA · RETO BETSSON

Intelligent Game Validation
Validador automático de juegos de casino

Informe del proyecto del equipo EscuadronMCP

Repositorio	github.com/EscuadronMCP/BetssonEscuadronMCP
Mercado validado	betsson.es (español, EUR) · Casino → Pragmatic Play
Juegos demostrados	Sweet Bonanza 2500 y Big Bass Halloween Splash 1000
Resultado	**PASS** en los dos escenarios, en local y en GitHub Actions
Fecha	15 de octubre de 2026

> **En una frase:** el programa abre el navegador, entra en betsson.es, inicia sesión, busca un juego de Pragmatic Play, juega una tirada con dinero real y entrega un informe en verde o rojo, con capturas, de cada comprobación, **sin ninguna intervención humana** y mirando la pantalla como lo haría un jugador.

1. Resumen

- **Qué valida:** los dos recorridos del reto. *Tirada completada* (cuenta con saldo): el juego gira de verdad, la apuesta y la divisa son las mostradas y **saldo después = saldo antes – apuesta + premio**, en el juego y en Betsson. *Saldo insuficiente* (cuenta sin saldo): la tirada se rechaza, aparece el mensaje en español y ningún saldo cambia.
- **Cómo:** cada comprobación la confirman hasta **cinco testigos independientes** (pantalla del juego, movimiento de los rodillos, saldo de Betsson, servidor del juego y mensajes del juego a la web). La pantalla es obligatoria para un PASS y cualquier contradicción es FAIL.
- **Coste:** 0 € de IA por ejecución; la IA solo se usa una vez para aprender un juego nuevo (≈ 0,001–0,008 $ medidos).
- **Tiempo:** ≈ 70–95 s los dos escenarios, en paralelo. Informe de menos de 1 MB.
- **Entregables del reto:** código con configuración por mercado y sin secretos; workflow de GitHub Actions que ejecuta ambos escenarios y publica el informe; informe PASS/FAIL por comprobación, escenario y mercado con capturas, traza y vídeo opcional.

2. El reto y cómo lo cubrimos

Requisito oficial	Cómo se cumple
Navegar por el casino y lanzar un juego de Pragmatic Play en Real Play	Casino → Filtros → chip «Pragmatic Play» → Resultados → juego. Se acepta el límite de sesión obligatorio en España (5 horas, 100 %).
Validar a partir de lo visible en pantalla (el juego es un canvas en un iframe)	OCR local del marcador del juego (crédito, apuesta, premio, mensajes) y búsqueda de botones por su imagen. El DOM del juego no se usa.
Un clic en *Spin* no es prueba	Se mide el movimiento de los rodillos (empieza y vuelve al reposo) y se leen los saldos antes y después.
Escenario 1: saldo antes, apuesta y divisa, inicio y fin, resultado, saldos después	Cinco comprobaciones con la regla `después = antes − apuesta + premio` en el juego y en la cabecera de Betsson.
Escenario 2: rechazo, mensaje en el idioma esperado, ningún saldo cambia	Mensaje leído completo («Fondos insuficientes para realizar esta apuesta…»); el juego confirma el idioma `ES`; saldos iguales.
Configuración por mercado sin tocar código	`config/markets/es.yaml` (URL, idioma, divisa, textos de la web, variables de credenciales). Otro mercado = otro YAML.
Workflow de GitHub Actions por mercado	`validate.yml`: matriz por mercado, informe como artefacto y resumen en la ejecución. Runner propio con IP española (betsson.es bloquea otros países).
Automatización total, sin humanos	Del arranque del navegador al informe: cookies, login (con el antibots AWS WAF), filtros, sesión de juego, bienvenida, apuesta mínima, tirada e informe.

Sin secretos en repositorio ni evidencias	Credenciales solo en `.env` / GitHub Secrets; login difuminado; traza limpiada; escáner de texto y OCR de capturas que retira cualquier evidencia con secretos.

3. Enfoque: testigos independientes

Un auditor no se fía de una sola fuente. El validador tampoco: cada hecho se contrasta con varias fuentes que no dependen entre sí.

Testigo	Tipo	Qué aporta
Pantalla del juego (OCR)	Visual	Lo que ve el jugador: crédito, apuesta, premio, divisa y mensajes.
Movimiento de los rodillos	Visual	Que el juego giró y se detuvo (umbrales adaptativos al fondo animado de cada juego).
Cabecera de Betsson	Visual / corrobora	Saldo de la web antes y después; para la divisa solo corrobora (no sustituye al marcador del juego).
Servidor del juego	Corrobora	Tráfico observado sin modificarlo: apuesta **aceptada** por el servidor, premio declarado y saldo final.
Mensajes del juego a la web	Corrobora	Eventos `postMessage` del juego a Betsson (ronda, apuesta, premio, saldo, idioma), solo si los emite el propio marco del juego.

> **Regla del veredicto.** PASS solo si la pantalla confirma el hecho y ningún testigo lo contradice. Un testigo no disponible no impide el PASS, pero queda anotado. Una contradicción es **FAIL por discrepancia**: justo el fallo de integración que interesa a Betsson (el jugador ve una cosa y el sistema registra otra).

Perfiles aprendidos: se adapta a juegos y proveedores

Lo que cambia de un juego a otro (textos del marcador, aspecto de los botones, zona de rodillos) no está programado: una IA de visión lo aprende **una vez** y lo guarda como perfil en `profiles/`. Antes de aceptarlo se valida por su efecto (el crédito leído coincide con el saldo de Betsson, «–» baja la apuesta…). Después, cada ejecución es local y determinista, sin IA. Si el proveedor mueve un botón o cambia un texto, se reencuentra solo; si cambia por completo, se reaprende una vez y se anota en el informe.

Flujo de principio a fin

Abrir Chrome → betsson.es + cookies → Login → Saldo Betsson → Casino → Filtros → Pragmatic Play → Juego + sesión obligatoria → Apuesta mínima → Condiciones previas → Una tirada observada → Saldos finales → Informe

4. Arquitectura y calidad

Módulo	Responsabilidad
`casino/`	Web de Betsson: cookies, login, saldo, filtros, sesión de juego.

`game/`	Jugar mirando la pantalla (genérico) y grabar eventos del juego.
`perception/`	OCR, búsqueda por imagen, movimiento, IA de visión.
`providers/`	Testigos específicos de cada proveedor, en un registro.
`scenarios/`	Comprobaciones, condiciones previas y fallo de demostración.
`domain/`	Dinero, regla del veredicto, resultados (lógica pura).
`report/`	Informe HTML/JSON y resumen para GitHub.

- **SOLID:** añadir un mercado, un proveedor o un juego se hace con configuración o con una implementación nueva registrada; el ejecutor no conoce a ningún proveedor.
- **Inversión de dependencias:** OCR, IA y testigos se inyectan detrás de interfaces (`Protocol`).
- **Tipado estricto** (mypy), **ruff** y **174 tests unitarios** con capturas y tráfico reales; CI en cada push.
- **Errores explícitos:** un fallo esperado produce un FAIL con su motivo, nunca un cierre silencioso.
- **Decisiones documentadas** en `docs/decisiones/` (ADR 001–003).

5. Seguridad y dinero real

Riesgo	Medida
Credenciales en el código	Solo variables de entorno (`.env` ignorado por git) y GitHub Secrets; `.env.example` sin valores.
Credenciales o tokens en evidencias	Campos de login difuminados; traza ligera y limpiada; escáner de texto (incluye formas codificadas en URL, JSON y HTML) y **OCR de cada captura**; lo que no se puede limpiar se retira y el informe lo marca como «no disponible». El propio informe final se limpia y se audita.
Tokens en mensajes de error	Los errores se limpian con el mismo escáner antes de llegar al informe.
Gastar dinero por error	Apuesta mínima comprobada en pantalla, tope por proveedor, divisa del mercado, nunca se repite una tirada; **condiciones previas**: la cuenta con saldo debe poder pagar y la cuenta «sin saldo» no debe tener fondos (si los tiene, no se gira).
Cuentas cruzadas	Se rechaza ejecutar si las dos cuentas son la misma.
Carga sobre la web	Una tirada por escenario, esperas por condición (sin esperas fijas innecesarias), un único reintento de login si la web lo limita.

6. El informe: cómo se ve un PASS y un FAIL

Cada ejecución genera `index.html` (para personas), `report.json` (para máquinas) y las evidencias de cada escenario: capturas, eventos del juego, tráfico sin tokens, resumen de lo observado y traza. Cada fila dice qué se comprobó, el resultado, el motivo en lenguaje claro y qué dijo cada testigo, con sus capturas.

> **Fallo intencionado para los mentores.** La carpeta `mentores/` incluye un informe real en el que el escenario 1 pasa y en el escenario 2 se provocó **a propósito** un fallo (`run --inject-fault insufficient_balance`): al saldo final de Betsson leído se le suma 0,01 € antes de evaluar, como si Betsson cobrara algo pese a rechazar la tirada. El informe lo anuncia en un recuadro y marca **FAIL** la comprobación «El saldo de Betsson no cambia», con el motivo y las capturas del saldo real. Lo mismo puede lanzarse desde GitHub Actions con la opción *inject_fault*.

7. Resultados y verificación

Verificación	Resultado
Sweet Bonanza 2500, ambos escenarios (local)	**PASS** · ≈ 90 s · 0 llamadas a la IA
Big Bass Halloween Splash 1000, ambos escenarios (local)	**PASS** · perfil aprendido por ≈ 0,001 $
GitHub Actions (runner propio en España)	**PASS** en ambos escenarios en 74 s con el código final, informe publicado como artefacto
GitHub Actions con el fallo intencionado (*inject_fault*)	**FAIL** anunciado como intencionado; el informe se publica igualmente como artefacto
Fallo real detectado (15/10)	**FAIL por discrepancia** cuando otra sesión gastó 2,10 € de la cuenta durante la tirada: el validador lo detectó y lo explicó.
Sesión del juego expirada (15/10, 17:22)	**FAIL** «el juego no reaccionó al botón de girar»: el proveedor mostró «Sesión expirada» al pulsar girar. No se gastó dinero y no hubo PASS falso; desde entonces el motivo del informe cita ese aviso.
Calidad	Formato, lint, tipos y 174 tests en verde en CI.

8. Costes y rendimiento

Concepto	Coste / medida
Cada ejecución (juego ya conocido)	**0 € en IA**: OCR y visión en local
Incorporar un juego nuevo	≈ 0,001 $ (medido 0,0006–0,0080 $ con Claude Haiku/Sonnet)
Tirada de prueba	0,20 € de apuesta; ≈ 0,01 € de coste esperado (RTP ≈ 94,5 %)
Duración	≈ 70–95 s ambos escenarios (antes ≈ 3 min)
Peso del informe	< 1 MB (capturas JPEG justas; vídeo solo con `--video`)

9. Revisión crítica y correcciones

Antes de la entrega se hizo una auditoría interna del código buscando PASS falsos y riesgos con dinero real. Todo lo corregido quedó cubierto por tests de regresión:

- **Dinero real:** condiciones previas antes de girar (apuesta positiva, en la divisa del mercado y bajo el tope; fondos según el escenario) y cuentas distintas.
- **Fidelidad del servidor:** cuenta la apuesta *aceptada* en la respuesta del servidor, no la pedida; se usa el premio que el servidor declara y su propia cuenta debe cuadrar.
- **Eventos del juego:** solo se aceptan si los emite el marco del juego (atribución por ventana emisora, también dentro de shadow DOM); rondas repetidas, desordenadas o con apuestas distintas se marcan como incoherentes; valores no numéricos, NaN o infinitos se descartan.
- **Lectura de pantalla:** una etiqueta nunca toma el importe de otra; se descartan lecturas de poca confianza; se conserva el signo negativo; números separados no se fusionan.
- **Veredictos:** la divisa debe verse en el marcador del juego (la cabecera de Betsson solo corrobora); valores mezclados (EUR/USD, ES/EN) son FAIL; un resultado repetido en PASS no puede ocultar uno en FAIL; un mensaje de rechazo que ya estaba antes de girar no prueba nada.
- **Secretos:** formas escapadas en JSON/HTML, tokens en URLs de errores, OCR de capturas, archivos ilegibles retirados, informe final auditado y enlaces de Windows con formato de URL.

10. Limitaciones conocidas

- **IP española:** betsson.es solo responde desde España, por eso el workflow usa un runner propio. En producción, una máquina x86 en Oracle Cloud Madrid.
- **Otros proveedores:** la arquitectura está preparada (configuración y registro de testigos) y Play'n GO ya está configurado, pero su aprendizaje aún no se ha completado; la compatibilidad demostrada de principio a fin es Pragmatic Play.
- **Vídeo opcional:** el vídeo no se escanea con OCR (los campos de login se difuminan en origen); por eso no se graba por defecto.
- **Cuenta compartida:** si otra persona usa la cuenta con saldo durante una ejecución, el resultado es FAIL por discrepancia (correctamente).

11. Cómo ejecutarlo

Requisitos: `uv` y Google Chrome. Credenciales en `.env` (ver `.env.example`).

`uv sync` · `uv run python -m betsson_validator run` (ambos escenarios) · `run --inject-fault insufficient_balance` (demostración de un FAIL) · `learn --game <slug>` (juego nuevo). El informe queda en `reports/<fecha>/index.html`.